

Assignment Task 5 (Working)

Component A: Snakes and Ladders (1)

1. Transition probabilities for the most boring game

- to find the transition matrix P , we need to calculate the probability P_{ij} of moving from square i to square j for a 10 square game.
- each outcome of a 6 sided die has a probability of $1/6$
- for $i = 1, 2, 3$,
 - rolling a 1 through 6 will land on a $i+1$ to $i+6$ square on the board
 $\therefore$ row 1 will have a 1 in columns 2-7 and 0 everywhere else
- for $i = 4, 5, 6, 7, 8, 9$,
 - from square 4, rolling a 1, 2, 3, 4 or 5 lands us on squares 5, 6, 7, 8 or 9. rolling a 6 lands us on the 10th square.
 $\therefore$ columns 5-10 each get a 1
 - from square 5, rolling a 1, 2, 3 or 4 lands us on squares 5, 6, 7, or 8. rolling a 5 or 6 lands us on the 10th square.
 $\therefore$ column 10 gets a value of 2
- for $i = 10$,
 - once we reach the 10th position, the piece stays there regardless of what we roll
 $\therefore$ row 10 will have a 6 in the 10th column and zero everywhere else.

a little python code for it:

```
vs code > assignment task 5.py > ...
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  # constructing the transition matrix
5  P = np.zeros((10, 10))    #10x10 zero matrix
6
7  for i in range(9):
8      for die in range(1, 7):
9          target_square = i + die
10
11         #if we overshoot the 10th square, stop on the 10th square
12         if target_square > 9:
13             P[i, 9] += 1
14
15         else:
16             P[i, target_square] += 1
17
18  P[9, 9] = 6
19
20  print(P.astype(int))
21  plt.show()
```

output:

```
[[0 1 1 1 1 1 1 0 0 0]
 [0 0 1 1 1 1 1 1 0 0]
 [0 0 0 1 1 1 1 1 1 0]
 [0 0 0 0 1 1 1 1 1 1]
 [0 0 0 0 0 1 1 1 1 2]
 [0 0 0 0 0 0 1 1 1 3]
 [0 0 0 0 0 0 0 1 1 4]
 [0 0 0 0 0 0 0 0 1 5]
 [0 0 0 0 0 0 0 0 0 6]
 [0 0 0 0 0 0 0 0 0 6]]
```

2. Expected game duration:

- from the first square we can move to any of squares 2 through 7 (each with $1/6$ probability), but it takes us a move to do so.
 - rolling the die costs us exactly 1 turn
 - we look at all the places a single move could land us and multiply the probability of landing there by the expected time left from “new square”.
- for square 1, a die roll spreads our chances equally ($\frac{1}{6}$ each) across squares 2, 3, 4, 5, 6, and 7.
- taking h_i as the expected number of moves to finish the game starting from position i ,

$$h_1 = 1 + \frac{1}{6} h_2 + \frac{1}{6} h_3 + \frac{1}{6} h_4 + \frac{1}{6} h_5 + \frac{1}{6} h_6 + \frac{1}{6} h_7$$

- now, we have to find the expected number of moves to finish the game from each of the positions 2 through 7, and so on.
 - we know that if we start in position 10 then we've won immediately, in 0 moves.
- $$h_{10} = 0$$
- trying to turn this situation into a systematic equation
 - h = destination target squares = a column holding our unknowns (h_1-h_9)
 - s = step = column vector of all 1s. ensures that no matter which square (1 through 9) we start our turn on, we accumulate that +1 turn cost for rolling the die.
 - $\tilde{P}$ = the transition probabilities
 - = when we multiply it by the vector h , it calculates the weighted averages of the remaining steps for every row at the same time.

this gives:

$$h = s + \tilde{P} h$$

3. Solving for the expected game duration:

- rearranging the correct option from the last question, we end up with the following working that leads us to a key equation:

$$h = s + \tilde{P} h$$

$$Ih = s + \tilde{P} h$$

$$(I - \tilde{P}) h = s$$

running the code below,

```
# PART 2&3
P_prob = P / 6.0

P_tilde = P_prob[0:9, 0:9]
s = np.ones((9, 1))
I = np.eye(9)

h = np.round(np.linalg.solve(I - P_tilde, s), 2)
print("h =\n", h)

print("h flattened =\n", h.flatten())
```

we get,

```
h flattened =  
[3.04 2.78 2.52 2.16 1.85 1.59 1.36 1.17 1. ]
```

4. Including snakes and ladders into the game

- following the below rules:
 - if our piece ends its movement on the starting square i of a snake or ladder, the next turn, you move to the end square j of the snake or ladder (instead of rolling).
 - its a snake if $i > j$
 - its a ladder if $i < j$
 - there can't be a ladder starting at the first square.
 - there can't be a snake starting at the last square.
 - each square can have at most one start or end of a snake or ladder (so you can't form chains or loops, or have multiple things start at the same place)
 - you can't put the start of six snakes on consecutive squares ($\because$ it will be impossible to get past the snakes, and the game won't end).
- reasoning:
 - rows represent where we **start** our turn
 - columns represent where we **land**
 - if a snake or ladder starts at square i , it completely changes our options when we are standing on the square i
 $\therefore$ the entire behaviour of i^{th} row changes
 - the rule set states that when we are on a square i , we move to square j instead of rolling.
this means all our normal die-rolling probabilities for row i disappear and we no longer have a $\frac{1}{6}$ chance of moving to different squares
 $\therefore$ 100% chance of moving to j .

$\therefore$ **the probability of moving from i to j becomes 1 (100%)**

$\therefore$ **the probability of moving from i to any other square becomes 0**

so in linear algebra terms,

because a transition matrix flattens its data horizontally, we transpose the column vector into a horizontal row vector.

5. Putting it all together

- choosing values for the SL vector (following the criteria):

- no ladder starts at 1
- no snake starts at 25 (or no win)
- no chains or loops: because no number is repeated, a player can never trigger both a ladder and snake at the same time.
- no 6 consecutive snakes
- adding fun to it 😊
 - 3 consecutive ladders placed towards the start of the board for a “boost”.
 - around the doorstep to victory, box 22, 23, 24, i'll place massive snakes to plummet all the way back down.
- placing the values into matlab:

```

1  % Enter the SL vector below
2  SL = [2 16; 3 15; 4 14; 22 5; 23 6; 24 7]
3
4
5  % We have included some Matlab code below. You do not need to understand it all to answer the question, but
6  % we encourage you to look at the loops and ask for help if you do not see how they work. It is very common
7  % to use loops in this sort of project.
8
9  % The number of squares
10 n = 25;
11
12 % Construct the P matrix first by ignoring the snakes and ladders
13
14 % The probability of taking a step from i to j is determined by the die
15 P = zeros(n);
16 for i=1:n
17     for j=i+1:min(i+6,n)
18         P(i,j) = 1/6;
19     end
20 end
21
22 % Consider the case when the step could be greater than the number
23 % of places remaining on the board
24 for i = 1:6
25     P(n-6+i, n) = min((i+1)/6, 1);
26 end
27
28 % Adjust the probability when landing on the start of a ladder
29 % or tail of a snake
30 no_SL = height(SL);
31 for i= 1:no_SL
32     for j=1:n
33         if j == SL(i,2)
34             P(SL(i,1),j) = 1;
35         else
36             P(SL(i,1),j) = 0;
37         end
38     end
39 end
40
41 % Construct the s vector
42 s = ones([n-1 1]);
43
44 % Extract tildeP
45 tildeP = P(1:(n-1),1:(n-1));
46
47 % Solve for h below
48 I_mat = eye(n-1);
49 h = (I_mat - tildeP) \ s
50
51 % Plot the expect game duration

```

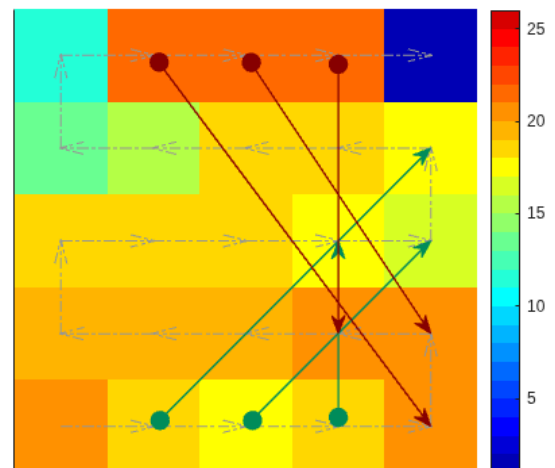
- SL output

SL =	
2	16
3	15
4	14
22	5
23	6
24	7

- h vector output

```
h =
20.3725
18.5647
17.8939
18.7647
20.5895
20.3461
20.0762
19.7460
19.3385
19.0851
18.9451
18.8857
18.4566
17.7647
16.8939
17.5647
18.1049
18.5293
15.8823
13.6134
11.6686
21.5895
21.3461
21.0762
```

- plot output



fun little criss-cross pattern vertically across the board. basically a closely* infinite loop whilst following the rules. adds a little kick to frustration build up.

*terms and conditions applied

Component B: Snakes and Ladders (2)

1. Where will we end up after 2 turns?

- starting with the same board of 25 squares, but with no snakes and ladders.

- lets look at the probability of each path
 - $1 > 2 > 7$
 - to go from 1 to 2, must roll a 1 (prob = $\frac{1}{6}$)
 - to go from 2 to 7, must roll a 5 (prob = $\frac{1}{6}$)
 - total for this path = $\frac{1}{6} \times \frac{1}{6} = 1/36$
 - $1 > 3 > 7$
 - to go from 1 to 3, must roll a 2 (prob = $\frac{1}{6}$)
 - to go from 3 to 7, must roll a 4 (prob = $\frac{1}{6}$)
 - total for this path = $\frac{1}{6} \times \frac{1}{6} = 1/36$
 - $1 > 4 > 7$
 - to go from 1 to 4, must roll a 3 (prob = $\frac{1}{6}$)
 - to go from 4 to 7, must roll a 3 (prob = $\frac{1}{6}$)
 - total for this path = $\frac{1}{6} \times \frac{1}{6} = 1/36$
 - $1 > 5 > 7$
 - to go from 1 to 5, must roll a 4 (prob = $\frac{1}{6}$)
 - to go from 5 to 7, must roll a 2 (prob = $\frac{1}{6}$)
 - total for this path = $\frac{1}{6} \times \frac{1}{6} = 1/36$
 - $1 > 6 > 7$
 - to go from 1 to 6, must roll a 5 (prob = $\frac{1}{6}$)
 - to go from 6 to 7, must roll a 1 (prob = $\frac{1}{6}$)
 - total for this path = $\frac{1}{6} \times \frac{1}{6} = 1/36$
- summing all probabilities = $5 \times 1/36 = 5/36$
 - $\therefore$ the probability of ending up on square 7 at the end of turn two is $5/36$

2. Where will we end up after 2 turns, but better

- so doing individual calculations is easy but annoying
- lets think of the matrix P as a flight schedule
 - the **row** is where we start
 - the **column** is where we want to land
- if we want to know the prob of moving from square 1 to square j in one single turn, we just look at row 1, column j of the matrix P. we can write this location as $(P)_{1j}$
- what happens when we take 2 turns? we multiply the matrix P by itself to get P^2 , which is basically telling the math to calculate every possible 2-step flight connection automatically. so instead of manually writing out all possible paths like before, we just look inside the P^2 matrix $(P^2)_{1j}$
- hence option 1 is right.

3. Where did we end up?

- like said above, each turn represents an independent die roll- so, multiplying the transition matrix by itself gives every combination of the paths.

•
• •

```
% Evaluate the product:
P_two_turns = P * P
```

- we are starting on square 1, so our row index is 1
- we want to know the probability of all 25 squares

```
% Extract the appropriate row
Ans = P_two_turns(1, :)
```

- full code:

- output:

```
Ans =
```

Columns 1 through 12											
0	0	0.0278	0.0556	0.0833	0.1111	0.1389	0.1667	0.1389	0.1111	0.0833	0.0556
Columns 13 through 24											
0.0278	0	0	0	0	0	0	0	0	0	0	0
Column 25											
0											

4. More steps into the future

- we can apply the same logic we used for the two turns and scale it up to any number of turns.

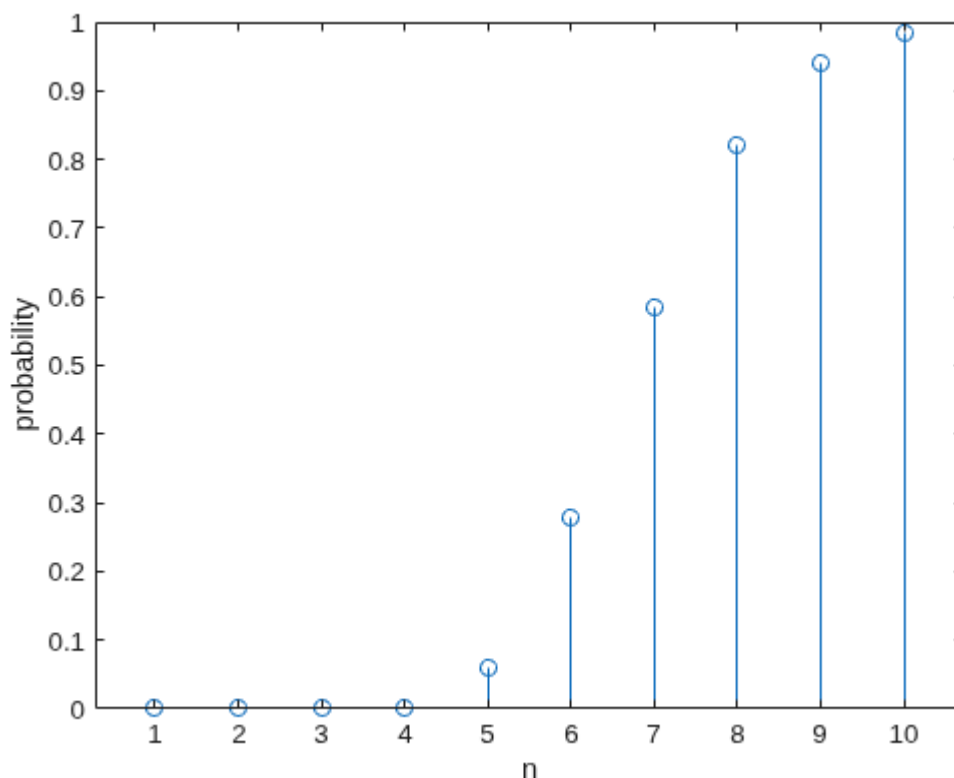
$$(P^n)_{ij} \text{ - option 2}$$

5. Probability of finishing the game by turn n

- we need to track the probability of ending up on square 25 🏆 after exactly n turns.
- since we're starting from square 1, we need to look at row 1, column 25 of the transition matrix raised to the power of n .
- so we define:

```
% Your value for the probability of winning by the end of turn n goes below
P_n_turns = P^n
Ans = P_n_turns(1, 25);
```

- we get the following plot:



- a couple of interesting observations:
 - the probabilities for the first three turns are exactly 0. since the maximum roll on a single die is a 6, the furthest possible square we

can hit in three turns is Square 19 ($1 + 6 + 6 + 6$). we are physically locked out of winning until turn 4.

- looking at $n = 4$, the line is just barely hovering above the zero mark. this visually says how rare it is to win in exactly 4 turns as it requires a perfect streak of four consecutive 6s ($(\frac{1}{6})^4 = 1/1296$). roughly a 0.077% chance.
- once we hit turn 5 and so on, the probability skyrockets exponentially. by turn 7, we have around a 60% chance of having completed the game. by turn 10, the probability is practically at 100% = a standard 25-square board game with no obstacles is almost guaranteed to end within 10 rolls. 🎲

6. Including some snakes and ladders

- goal: make it extremely unlikely to win by turn 10 while following the below rules:
 - no ladder starting at square 1
 - no snake starting at square 25
 - no chains or sharing squares
 - no 6 consecutive snake heads in a row
- SL vector:
 - towards the finish line, i'll add 2 large snakes at squares 23 and 24 ($[23\ 3], [24\ 2]$), where the player is most likely to land during their final rolls. this will drop the player to the beginning of the board.
 - even if the player avoids the top row traps, i'll have a second wall of defense at squares 18 and 17, and a third wall at 12 and 11 ($[18\ 4], [17\ 5], [12\ 6], [11\ 7]$)
 - so,

```
% Enter SL below
SL = [24 2; 23 3; 18 4; 17 5; 12 6; 11 7]
```

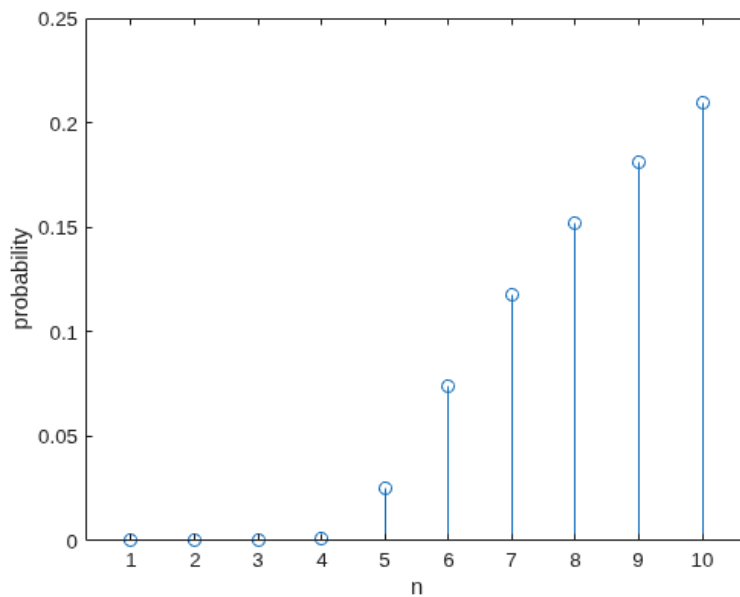
- updating the answer:

```
% Your value for the probability of winning by the end of turn n goes below
P_n_turns = P^n
Ans = P_n_turns(1, 25);
```

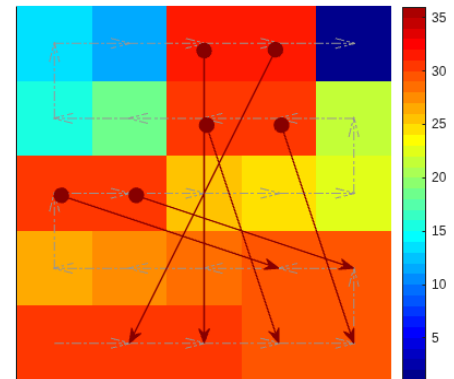
- final code:

```
1 % We build the matrix P and you are required to use P to calculate the probability of winning by the
2 % end of turn n. Enter your solution in Ans below. Note that sol records the answer for different values of
3 % n and is then used to plot the graph
4
5 % Enter SL below
6 SL = [24 2; 23 3; 18 4; 17 5; 12 6; 11 7]
7
8 % Setting up P
9
10 % Construct P, ignoring the snakes and ladders
11 P = zeros(25);
12 for i=1:25
13     for j=i+1:min(i+6,25)
14         P(i,j) = 1/6;
15     end
16 end
17 for i = 1:6
18     P(25-6+i, 25) = min((i+1)/6, 1);
19 end
20
21 % Adjust for the snakes and ladders
22 no_SL=height(SL);
23 for i= 1:no_SL
24     for j=1:25
25         if j == SL(i,2)
26             P(SL(i,1),j) = 1;
27         else
28             P(SL(i,1),j) = 0;
29         end
30     end
31 end
32
33 % Record the probability for different values of n
34 sol = [];
35 for n=1:10
36
37     % Your value for the probability of winning by the end of turn n goes below
38     P_n_turns = P^n
39     Ans = P_n_turns(1, 25);
40
41     sol = [sol, Ans];
42 end
43
44 % Plot it out
45 stem(sol);
46 xlabel('n')
47 ylabel('probability')
```

- output plot



- extra “heatmap” plot



- observations:

- the vertical scale of the graph has changed drastically. our probability at turn 10 drops all the way down to just ~0.21.
- at $n = 1, 2, 3, 4$, the probability of winning remains flat at exactly 0. even though the theoretical fastest route on an empty board is 4 turns, our snakes completely block those specific paths, meaning a 4-turn victory is now impossible.
- at $n = 5$ to 10, our board severely suppresses the curve (normally shoots up after 5). at turn 6, a player only has a 7.5% chance of winning.